LAPORAN CHECKPOINT QUESTION GRAF SAINS DATA



Disusun Oleh :

Alin Tifana Safitri	23031554019
Firly Farah Aulia	23031554047
Shazrin Fhariza Amerta	23031554150
Jesika Trisyanda	23031554228

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
S1 SAINS DATA
UNIVERSITAS NEGERI SURABAYA
2026

BAB I
CHECKPOINT QUESTIONS

Pertanyaan 1

Apa nama database yang digunakan pada tutorial ini?

Jawaban:

Nama database yang digunakan pada tutorial ini adalah neo4j. Database tersebut berfungsi untuk menyimpan dan mengelola struktur graf pertemanan, di mana objek entitas individu disimpan sebagai node Person dan dihubungkan oleh relasi :KNOWS.

Dalam skrip Python yang dijalankan, database ini digunakan oleh fungsi add_friend untuk membentuk simpul pertemanan (seperti merepresentasikan hubungan antara node "Arthur" dengan "Guinevere", "Lancelot", dan "Merlin"), serta digunakan oleh fungsi print_friends untuk membaca dan menampilkan daftar relasi tersebut secara terurut.

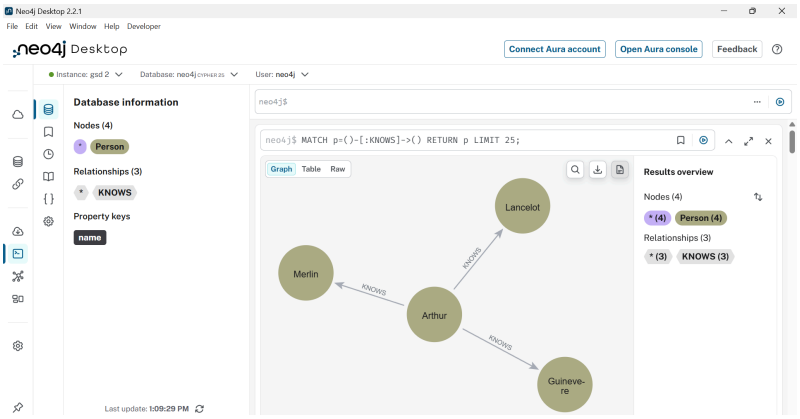
```python
from neo4j import GraphDatabase, RoutingControl

URI = "neo4j://127.0.0.1:7687"
AUTH = ("neo4j", "password")

def add_friend(driver, name, friend_name):
    driver.execute_query(
        "MERGE (a:Person {name: $name}) "
        "MERGE (friend: Person {name: $friend_name})"
        "MERGE (a)-[:KNOWS]->(friend)",
        name=name, friend_name=friend_name, database_="neo4j",
    )

def print_friends(driver, name):
    records, = driver.execute_query(
        "MATCH (a: Person)-[:KNOWS]->(friend) WHERE a.name = $name"
        "RETURN friend.name ORDER BY friend.name",
        name=name, database_="neo4j", routing_=RoutingControl.READ,
    )
    for record in records:
        print(record["friend.name"])
```

Setelah mendefinisikan string koneksi ke database neo4j, berikut adalah tampilan pada Neo4j desktop.

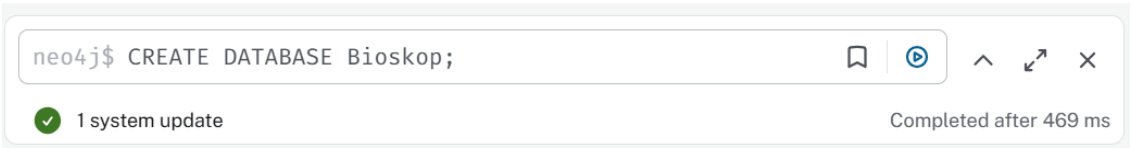


Pertanyaan 2

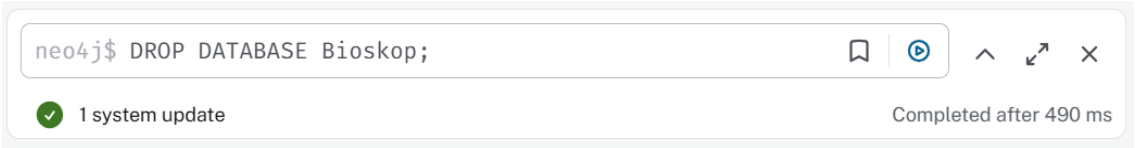
Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!

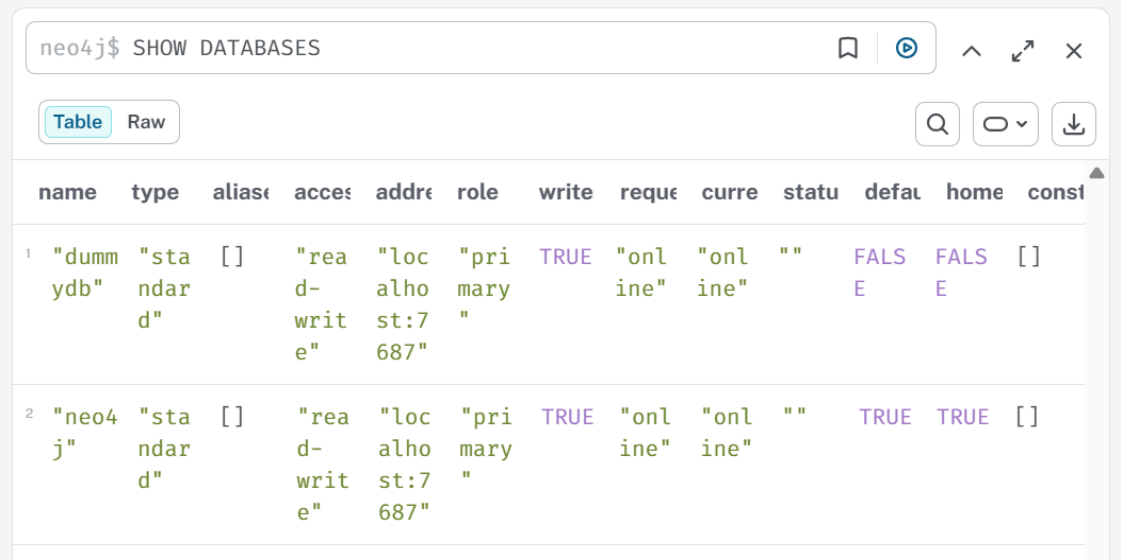
Jawaban:

1. DDL (Data Definition Language): CREATE DATABASE dan DROP DATABASE untuk membuat dan menghapus database.







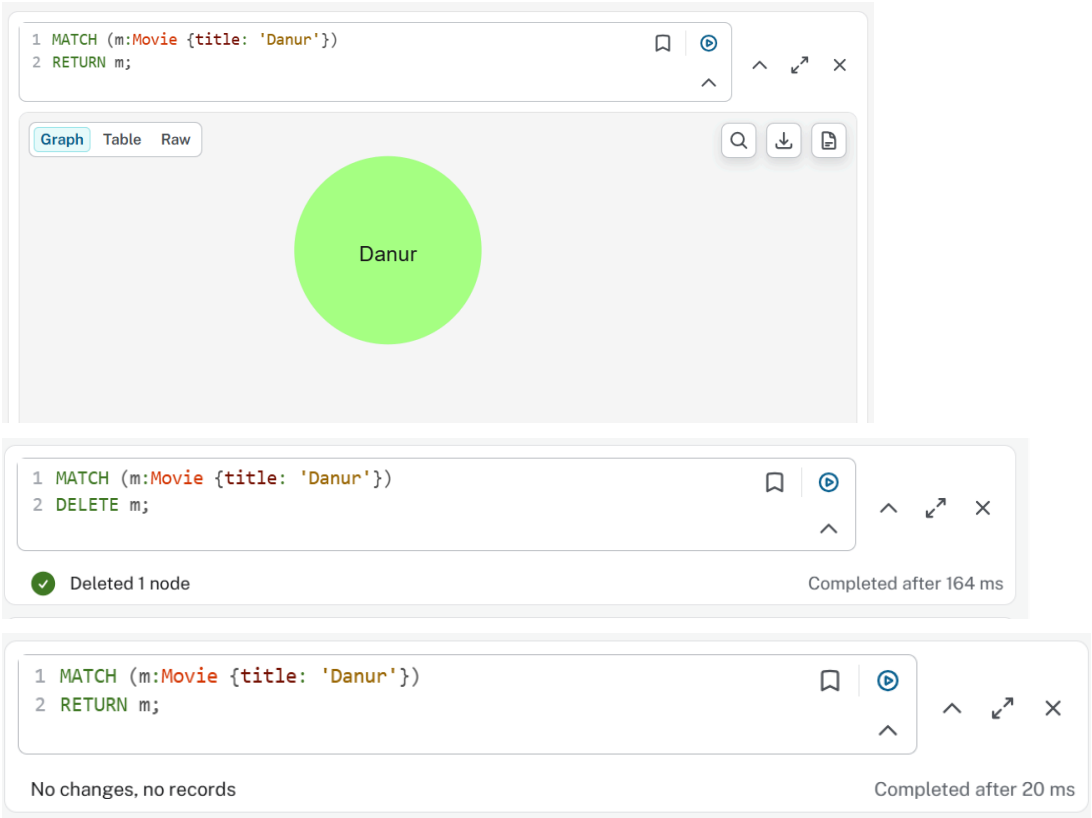


Perintah CREATE DATABASE digunakan untuk membuat database baru dengan nama Bioskop, sedangkan perintah SHOW DATABASES digunakan untuk menampilkan daftar database yang ada pada satu instance Neo4j. Setelah database Bioskop berhasil dibuat, database tersebut muncul pada daftar database.

Perintah DROP DATABASE Bioskop digunakan untuk menghapus database Bioskop. Setelah perintah tersebut dijalankan, database Bioskop tidak lagi muncul pada daftar database. Menunjukkan bahwa perintah DDL pada Neo4j dapat digunakan untuk membuat dan menghapus database serta memeriksa daftar database yang tersedia

2. DML (Data Manipulation Language): CREATE, MATCH, SET, dan DELETE untuk menambah, membaca, mengubah, dan menghapus data.

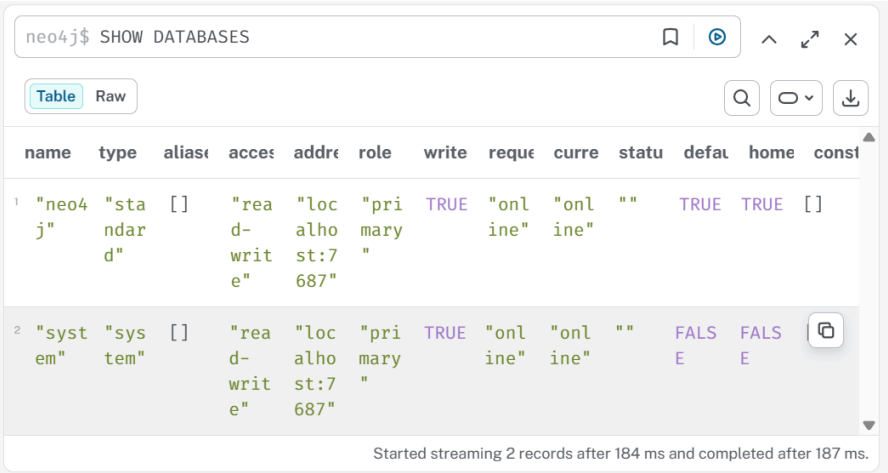




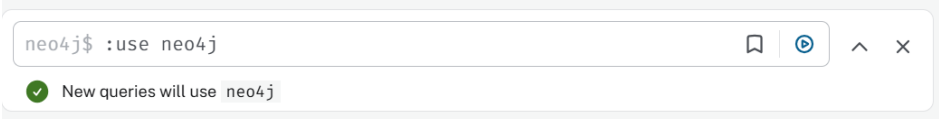
Perintah CREATE digunakan untuk menambahkan node film dengan label Movie beserta properti judul "Danur" dan tahun rilis 2015. Perintah MATCH selanjutnya dijalankan untuk mencari dan menampilkan data film tersebut. Hasil visualisasi menunjukkan node film "Danur" berhasil dibuat dan tersimpan di dalam database.

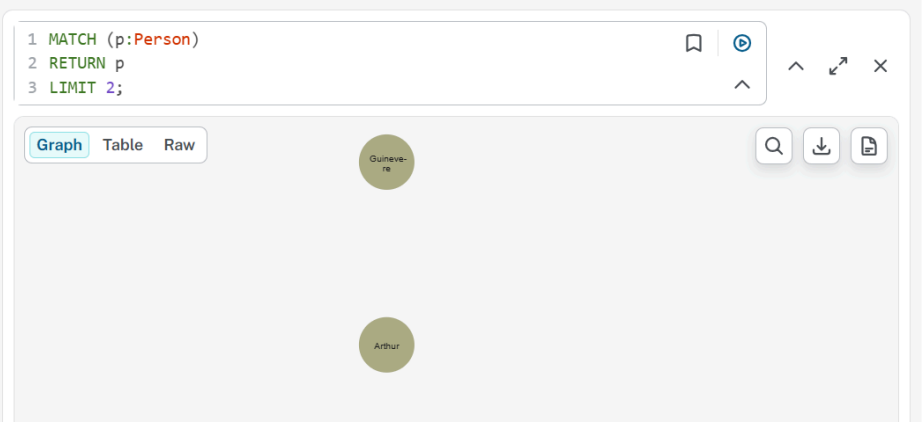
Perintah DELETE kemudian dieksekusi untuk menghapus node film "Danur". Eksekusi ulang perintah MATCH setelah penghapusan menghasilkan keterangan No changes, no records, yang mengindikasikan bahwa node film tersebut sudah tidak ditemukan. Berdasarkan alur tersebut, perintah DML pada Neo4j terbukti dapat digunakan untuk menambah, membaca, dan menghapus data secara efektif.

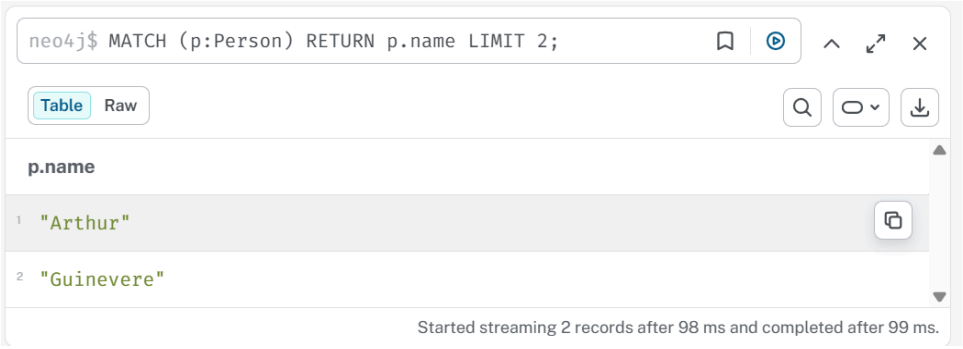
3. SHOW DATABASES: menampilkan daftar database yang tersedia dalam satu instance.

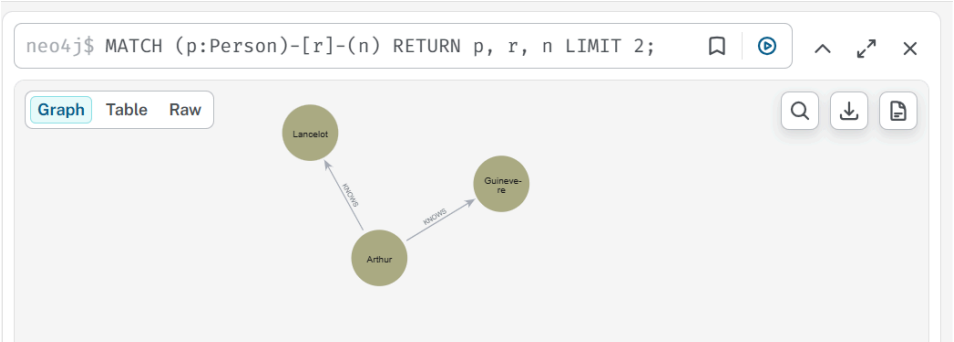


4. USE nama_database: memilih database yang akan digunakan untuk menjalankan query.









Perintah :use neo4j digunakan untuk memilih database neo4j sebagai database aktif untuk menjalankan query berikutnya. Setelah database dipilih, perintah MATCH (p:Person) RETURN p LIMIT 2 digunakan untuk menampilkan dua node dengan label Person. Hasil visualisasi menunjukkan dua node, sedangkan query MATCH (p:Person) RETURN p.name LIMIT 2 menampilkan nama orang, yaitu Arthur dan Guinevere.

Query MATCH (p:Person)-[r]-(n) RETURN p, r, n LIMIT 2 digunakan untuk menampilkan node Person beserta relasi dan node lain yang terhubung. Dengan demikian, praktikum ini menunjukkan bahwa pemilihan database dapat dilanjutkan dengan query untuk menampilkan data dan relasi dalam bentuk tabel maupun graf. Dengan demikian, perintah :use neo4j digunakan untuk memilih database neo4j sebagai database aktif sehingga query yang dijalankan setelahnya akan mengakses data pada database tersebut.

Pertanyaan 3

Apa jenis graf pertemanan yang telah dibuat di sini?

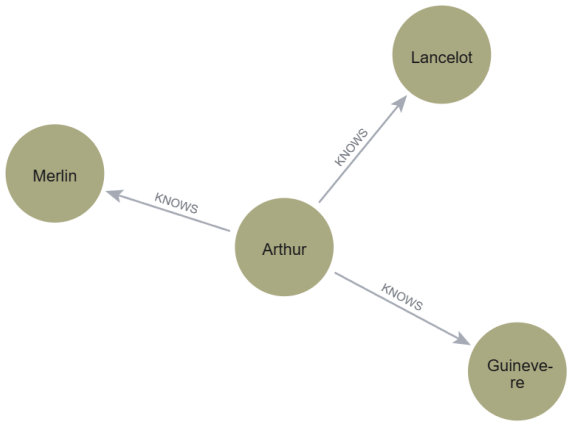
Jawaban:

Jenis graf pertemanan yang telah dibuat adalah directed graph atau graf berarah dengan label node Person dan relasi KNOWS

Relasinya ditunjukkan melalui sintaks:

```
"MERGE (a)-[:KNOWS]->(friend)"
```

Tanda panah -> menunjukkan arah hubungan dari satu orang ke orang lainnya. Arthur menjadi titik awal karena pada kode, nama Arthur ditulis sebagai orang pertama yang dimasukkan ke fungsi add_friend(), sedangkan Guinevere, Lancelot, dan Merlin menjadi orang yang dikenalnya. Jadi, hubungan yang terbentuk mengarah dari Arthur ke ketiga orang tersebut. Hubungan ini hanya menunjukkan bahwa Arthur mengenal mereka, bukan sebaliknya, karena kode belum membuat hubungan dari Guinevere, Lancelot, dan Merlin kembali ke Arthur.



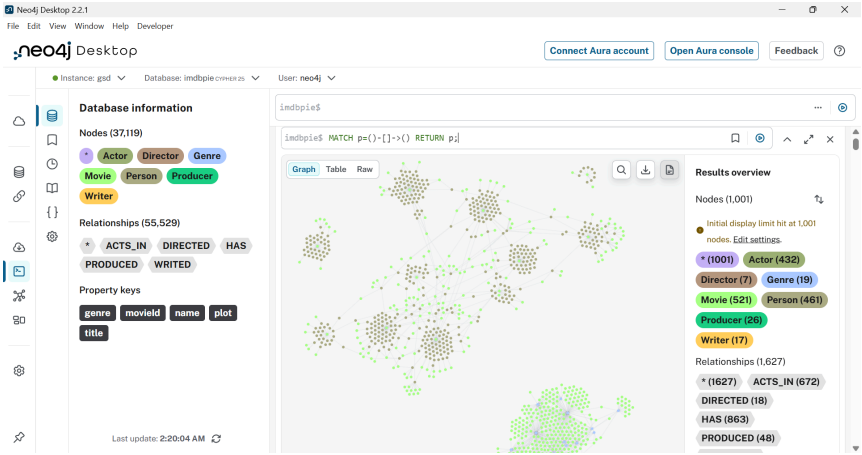
Pertanyaan 4

Carilah 1 pustaka Python selain: IMDBPy dan IMDBInfo yang dapat digunakan untuk mengakses API IMDB!

Jawaban:

Pustaka Python selain IMDbPY dan IMDbInfo yang awalnya dipilih untuk mengakses API IMDb adalah CinemagoerNG. Penerapan pustaka CinemagoerNG dalam proses impor detail film mengalami kegagalan total akibat pemblokiran akses oleh server IMDb yang mengembalikan status HTTP Error 403 (Forbidden). Pengambilan data selanjutnya dialihkan menggunakan pustaka imdbpie sebagai solusi alternatif. Pustaka imdbpie terbukti berhasil mengambil seluruh detail informasi film seperti sutradara, aktor, penulis, produser, genre, hingga plot cerita secara lancar.

Penggunaan imdbpie menghasilkan cakupan data kredits yang jauh lebih mendalam hingga ke level produser dan penulis. Kondisi ini membentuk graf dengan jumlah node Person yang jauh lebih besar dan terhubung utuh dengan node Movie dan Genre melalui relasi spesifik di Neo4j.



Pertanyaan 5

Pustaka IMDBpy membutuhkan pembaharuan, seperti yang dicantumkan pada laman github ini. Selain proses import menggunakan API IMDB, adakah sumber lain (exported dataset) yang dapat dipakai untuk? Sebutkan!

Jawaban:

Selain menggunakan API IMDb, dapat menggunakan **IMDb Non-Commercial Datasets** yang tersedia di https://datasets.imdbws.com/. Dataset ini dapat diunduh dalam format TSV dan berisi informasi film, seperti judul, tahun rilis, genre, rating, sutradara, dan pemeran. Dataset tersebut dapat digunakan sebagai sumber alternatif untuk proses impor data film ke Neo4j tanpa harus mengambil data melalui API IMDb.

Pertanyaan 6

Lakukan modifikasi seperlunya pada source code yang telah diberikan dengan mengadopsi workaround yang disebutkan pada link di soal (b)!

Jawaban:

Modifikasi dilakukan pada file movieParser.py dari library Cinemagoer dengan mengacu pada solusi alternatif (*workaround*) yang dibahas dalam GitHub Issue #537.

Workaround tersebut dilakukan dengan menyesuaikan XPath pada bagian pengambilan informasi film karena adanya perubahan struktur halaman IMDb.

Terdapat dua bagian kode yang dimodifikasi:

a. **Bagian Genre**

XPath sebelumnya:

```
//td[starts-with(text(), "Genre")]/..//li/a
```

Diubah menjadi:

```
//span[starts-with(text(), "Genre")]/..//li/a
```

Perubahan ini bertujuan menyesuaikan pencarian elemen Genre dengan struktur HTML IMDb yang baru.

b. **Bagian Director**

XPath sebelumnya:

```
//div[starts-with(normalize-space(text()), "Director")]/ul/li[1]/a
```

Diubah menjadi:

```
//li[@data-testid="title-pc-principal-credit"]/span[starts-with(text(), "Director")]/..//ul//a',
```

Perubahan ini bertujuan menyesuaikan pencarian informasi sutradara berdasarkan elemen data-testid pada halaman IMDb. Modifikasi dilakukan dengan mengganti XPath lama menggunakan XPath dari workaround yang dibahas pada Issue #537. Dengan perubahan ini, kode parser disesuaikan dengan struktur halaman IMDb yang digunakan dalam workaround tersebut. Pengujian pengambilan data belum berhasil diselesaikan karena koneksi ke server IMDb terputus.